

Symbolic Execution with KLEE and Static Analysis on WSL: A Step-by-Step Guide

1. Installing KLEE (Symbolic Execution Engine) on WSL

Setting up KLEE on Windows Subsystem for Linux (WSL) involves installing the necessary compiler toolchain, solver, and libraries. We will use an Ubuntu distribution on WSL (e.g. Ubuntu 22.04 on WSL2) for the following steps:

1. **Install System Dependencies:** Update packages and install development tools and libraries required by LLVM and KLEE ¹. This includes build tools (gcc, g++, make, cmake), 32-bit support libraries, and various dev libraries. For example:

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y build-essential cmake curl file git \
  g++-multilib gcc-multilib libcap-dev libncurses5-dev libsqlite3-dev \
  libtcmalloc-minimal4 libgoogle-perftools-dev unzip graphviz doxygen \
  python3-pip python3-tabulate
```

The above installs compilers, cmake, multilib support, profiling/tcmalloc libraries, SQLite3, and documentation tools ¹. We also install Python tools: `lit` (for running tests) and `wllvm` (to easily generate LLVM bytecode) via pip ²:

```
sudo pip3 install lit wllvm
```

Ensure that `~/local/bin` is in your PATH so the `wllvm` tool is accessible ³.

1. **Install LLVM/Clang:** KLEE works on LLVM bytecode, so we need LLVM and Clang. It's recommended to use LLVM 13 for the latest KLEE version ⁴. On Ubuntu, you can install a specific LLVM release from the official apt.llvm.org repository ⁵. For example, to install LLVM/Clang 13:

```
# Add LLVM repository (key and apt source for your Ubuntu version)
wget -O - https://apt.llvm.org/llvm-snapshot.gpg.key | sudo apt-key add -
sudo apt-add-repository "deb http://apt.llvm.org/jammy/ llvm-toolchain-
jammy-13 main"
sudo apt-get update
# Install clang-13 and LLVM 13 packages
sudo apt-get install -y clang-13 llvm-13 llvm-13-dev llvm-13-tools
```

This will provide `clang-13`, `clang++-13`, and corresponding `llvm-config-13` etc. (If you're on a different Ubuntu release, adjust the repository URL accordingly.) **Note:** You may set up `update-alternatives` or symlinks if needed so that `clang` in PATH points to `clang-13`, or specify `clang-13` explicitly in build steps.

2. **Install an SMT Solver (STP or Z3):** KLEE relies on an SMT solver for constraint solving. You must install at least one solver ⁶. The two most common options are:

3. **Z3:** A modern solver supported by KLEE ⁷. Z3 is easy to install via package manager. For example:

```
sudo apt-get install -y z3 libz3-dev
```

This installs the Z3 solver and its libraries (ensure the version is ≥ 4.4 , which is usually the case on recent Ubuntu).

4. **STP:** The original solver used by KLEE. If you prefer STP, you will need to build it from source. This involves installing its dependencies (`cmake`, `bison`, `flex`, `libboost-all-dev`, etc.) and the Minisat library, then compiling STP. For instance, KLEE documentation suggests building STP 2.3.3 with commands like:

```
# Install STP dependencies
sudo apt-get install -y cmake bison flex libboost-all-dev python perl
zlib1g-dev
# Build Minisat (SAT solver dependency for STP)
git clone https://github.com/stp/minisat.git && cd minisat
mkdir build && cd build
cmake -DSTATIC_BINARIES=ON -DCMAKE_INSTALL_PREFIX=/usr/local ../
sudo make install
cd ../../
# Build STP
git clone https://github.com/stp/stp.git && cd stp
git checkout tags/2.3.3 # use a stable release
mkdir build && cd build
cmake -DBUILD_SHARED_LIBS=OFF -DENABLE_PYTHON_INTERFACE=OFF ..
make -j4
sudo make install
```

(The above disables shared libs and the Python interface to avoid issues ⁸.) After installing STP, remember to set `ulimit -s unlimited` in your shell (or `limits.conf`) because STP can require a larger stack size ⁹.

You can choose **either** solver; KLEE supports both. For simplicity, we proceed with Z3 in this guide. KLEE will detect Z3 during its build if `libz3-dev` is installed. (If both solvers are present, you can enable one or both in KLEE's build options.)

1. **Build KLEE-uClibc (optional but recommended):** By default, KLEE assumes the analyzed program is "closed" (no external library calls). To analyze programs that use C library functions or system calls

(file I/O, `printf`, etc.), KLEE provides a POSIX runtime with a modeled C standard library (based on uClibc). We should compile **klee-uclibc**, which is a modified uClibc version for KLEE ¹⁰. Run the following:

```
git clone https://github.com/klee/klee-uclibc.git
cd klee-uclibc
./configure --make-llvm-lib \
    --with-cc=clang-13 \
    --with-llvm-config=llvm-config-13
make -j4
cd ..
```

This fetches the uClibc source and configures it to build bitcode libraries (`--make-llvm-lib`). We explicitly specify `clang-13` and `llvm-config-13` if the default `clang` isn't version 13 ¹¹. After a successful `make`, KLEE-uClibc builds a bitcode archive of the C standard library. We will later tell KLEE where to find this.

2. **Build and Install KLEE:** Now fetch the KLEE source code and compile it:

```
git clone https://github.com/klee/klee.git
cd klee
mkdir build && cd build
```

Configure KLEE with CMake. We need to enable the solver and uClibc we installed. For example, if using Z3 and uClibc:

```
cmake .. \
    -DENABLE_SOLVER_Z3=ON -DENABLE_SOLVER_STP=OFF \
    -DENABLE_POSIX_RUNTIME=ON \
    -DKLEE_UCLIBC_PATH=../klee-uclibc \
    -DLLVM_CONFIG_BINARY=/usr/bin/llvm-config-13
```

Explanation:

3. `-DENABLE_SOLVER_Z3=ON`: enable Z3 solver support in KLEE (and disable STP here since we choose Z3).
4. `-DENABLE_POSIX_RUNTIME=ON` and `-DKLEE_UCLIBC_PATH=<path>`: include KLEE's POSIX runtime and point to the uclibc directory we built ¹².
5. We also point CMake to the correct LLVM tools (`llvm-config-13`). If CMake has trouble finding your specific LLVM version, you can explicitly set `-DLLVM_CONFIG_BINARY` or use `-DLLVM_DIR=/usr/lib/llvm-13/` as needed ¹³.

Note: By default, KLEE's build uses Google perftools `tcmalloc` for its memory allocator (for performance with large allocations). We installed `libtcmalloc-minimal4` and `libgoogle-perftools-dev` earlier

for this. If you prefer to use the standard allocator, add `-DENABLE_TCMALLOC=OFF` to the CMake options ¹⁴.

Now build KLEE:

```
make -j4
```

After a successful build, you can (optionally) run KLEE's test suites:

```
make systemtests    # run basic regression tests
make unittests      # run unit tests (if enabled during cmake)
```

Finally, install KLEE binaries (optional, you can also run from the build directory):

```
sudo make install
```

This will typically install `klee` and its tools under `/usr/local/bin` (or the prefix you specified). At this point, KLEE is ready to use ¹⁵. You may want to add `/usr/local/bin` to your PATH if it's not already.

2. Sample Vulnerable C/C++ Code for Security Testing

Next, we prepare a sample program in C (or C++) that contains security issues which we will analyze using static analysis and symbolic execution. Below is an example C program with an **intentional buffer overflow vulnerability** and insufficient input validation:

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char buffer[10];
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <input>\n", argv[0]);
        return 1;
    }
    // Vulnerability: copying without bounds check can overflow 'buffer'
    strcpy(buffer, argv[1]); // POTENTIAL BUFFER OVERFLOW
    printf("Hello, %s!\n", buffer);
    return 0;
}
```

What's wrong in this code? It reads a user-provided string (`argv[1]`) into a fixed-size stack buffer of 10 bytes using `strcpy` with no length check. If the input exceeds 9 characters (plus the null terminator), it will overflow the buffer, leading to a **stack-based buffer overflow**. This is a classic security bug. A static

analyzer would likely warn about the use of `strcpy` or that the destination buffer may be too small. We will see how KLEE can systematically explore inputs to find this overflow.

(You can create other examples as well: for instance, a program that reads from `stdin` into a small buffer without checking length, or an example with an integer overflow or use-after-free. KLEE is capable of detecting various memory errors (buffer out-of-bounds, use-after-free, null dereferences, etc.)¹⁶, and we can use static analysis to flag such issues too.)

3. Compiling the Code to LLVM IR (Bitcode)

KLEE operates on **LLVM Intermediate Representation (IR)**. We need to compile the above C code into LLVM bitcode. Since we installed Clang as part of LLVM, we can use it to compile to a `.bc` or `.ll` file:

- **Compile to LLVM bitcode (.bc):** Use clang with `-emit-llvm`:

```
clang-13 -emit-llvm -c -g -O0 sample.c -o sample.bc
```

This generates an LLVM bitcode file (`sample.bc`) from `sample.c`. We include `-g` for debug info (so KLEE error messages have line numbers) and `-O0` to avoid optimizations that might prune paths. (Optionally, add `-Xclang -disable-O0-optnone` to prevent Clang from marking the IR as non-optimizable at O0 level.)

- **(Optional) Human-readable IR (.ll):** If you want to see the LLVM IR, you can compile to an LLVM assembly file:

```
clang-13 -S -emit-llvm -O0 sample.c -o sample.ll
```

This `.ll` file can be opened in a text editor to inspect the IR. It's not needed for KLEE, but useful for understanding.

For multi-file programs, you would compile each source to bitcode and then link them using `llvm-link`. We installed **wllvm** earlier, which can also help by compiling a whole project and extracting a single bitcode file (it embeds bitcode in the object files). For example, you could compile with `wllvm` as your compiler and then run `extract-bc` on the final binary to get a bitcode. But for our simple example, a single compile command is enough.

Now we have `sample.bc`, which is the LLVM IR for our vulnerable program. We'll use this as input to both KLEE and static analysis tools.

4. Running KLEE for Symbolic Execution on the Program

With KLEE installed and our program compiled to bitcode, we can perform symbolic execution to automatically explore inputs and find the vulnerability. KLEE can run the program in a sandboxed symbolic environment and report errors such as invalid memory accesses.

Basic KLEE usage: To run KLEE, use the `klee` command on the bitcode file. However, our program expects a command-line argument (the input string). We need to tell KLEE to provide a **symbolic argument**. KLEE's **POSIX runtime** (enabled by `-posix-runtime` flag) allows us to specify symbolic command-line arguments, symbolic files, or symbolic standard input, etc. In this case, we'll use `--sym-arg` to generate a symbolic `argv[1]`.

Run KLEE as follows:

```
klee --posix-runtime --libc=uclibc sample.bc --sym-arg 20
```

Explanation: - `--posix-runtime` instructs KLEE to use its POSIX/Linux environment model (this is required when using `--sym-arg` or `--sym-stdin`, etc., and when our program uses standard C library calls) ¹⁷. - `--libc=uclibc` explicitly tells KLEE to use the uClibc-based environment (not strictly necessary if KLEE was built with POSIX/uClibc by default). - `sample.bc` is our program's bitcode. - `--sym-arg 20` tells KLEE to provide *one* symbolic command-line argument of length 20 bytes to our `main` function ¹⁸ ¹⁷. This means KLEE will pretend that `argv[1]` is a 20-byte array of symbolic values (the contents are initially unconstrained, so KLEE will explore different possible bytes). We choose length 20 to allow exploration of cases longer than our 10-byte buffer (so that an overflow is possible). KLEE will consider inputs of all lengths up to 20 (it treats unconstrained bytes and will also consider the possibility of null terminators at various positions, etc.).

When you run this, KLEE will execute the program symbolically, exploring different branches depending on the symbolic input. In our program, any input is accepted (there's no branch except the usage message), but the different *sizes* and *contents* of the symbolic argument will affect whether a buffer overflow occurs. KLEE systematically considers the possibilities: - If the symbolic argument's length (or effectively the bytes before a null terminator) is ≤ 9 characters, the `strcpy` might succeed without overflowing. - If it's ≥ 10 , `strcpy` will attempt to write past the end of `buffer`, which should be detected as a memory error.

KLEE output: KLEE will log its progress to the console and create an output directory (by default `klee-out-0` for the first run). It should detect the buffer overflow and treat it as a failed assert (KLEE treats out-of-bounds memory access as a reportable error). You will see a message like:

```
KLEE: ERROR: sample.c: ... memory error: out of bound pointer
```

KLEE will generate a test case that triggers this error. For example, it might produce a file `klee-out-0/test000001.ktest` corresponding to the crashing input. It also produces a `.err` file describing the error (e.g. `test000001.err` with the error type and location).

In our run, KLEE should generate at least two test cases: one that **does not** overflow (e.g., perhaps an input of size 5 or an empty string) and one that **does overflow** (the bug-triggering input). For instance, KLEE might output:

```
KLEE: done: completed paths = 2
KLEE: done: generated tests = 2
```

One of those tests will be flagged with the error. We can use the KLEE **ktest-tool** to inspect the generated input values:

```
ktest-tool klee-out-0/test000001.ktest
```

This will print the concrete bytes of the symbolic argument for that test. You could then rerun the original program with that input to confirm it crashes (e.g., `./sample "$(cat test000001.ktest)"` after converting the ktest to a raw string, or using `klee-replay` tool).

KLEE's ability to generate concrete failing inputs is powerful: it has essentially performed an automatic test generation that finds the buffer overflow. In this simple case, the bug is obvious, but in larger programs, KLEE can explore complex conditions. It detects various errors like invalid memory accesses, assertion failures, division by zero, etc., and produces tests for them ¹⁶ ¹⁹.

(For reference, KLEE in one case found memory-corruption bugs (like buffer overflows) in real-world code and generated inputs to trigger them ²⁰. When it finds an error, it halts that execution path and saves a test case. The user can then analyze the path or replay it outside KLEE.)

5. Guiding Symbolic Execution with Static Analysis

Static analysis tools (like Clang Static Analyzer or Infer) can quickly scan the program's source code for potential bugs without executing it. They often report **warnings** or **error traces** (a sequence of steps leading to a suspected bug). However, static analysis can produce **false positives** (reporting bugs that might not be truly reachable or exploitable) because it makes conservative approximations ²¹. This is where combining it with symbolic execution (KLEE) is very beneficial: we can use static analysis results to **focus** symbolic execution on likely problem areas, and then KLEE can attempt to confirm the bug and produce a concrete input for it ²².

Here's how we can integrate static analysis with our KLEE workflow:

1. **Run a Static Analyzer on the Code:** We use Clang's static analyzer as an example. Clang Static Analyzer can be run via the `scan-build` tool on our program:

```
scan-build -o static_report clang -g sample.c -o sample.bin
```

This will compile `sample.c` and produce an HTML/static analysis report in the `static_report` directory. In our case, the analyzer will likely warn: **"Potential buffer overflow in call to `strcpy`"**

(and indeed it should, because `strcpy` usage with a fixed buffer is a known hazard). The report will include a trace: e.g., "taking `argv[1]` of length 20 leads to overflow of buffer of size 10" (the analyzer symbolically reasons about an arbitrary input and flags the dangerous call).

Another option: You can try **Infer** (Facebook's static analyzer) or other tools to get bug reports. The concept is the same – identify a potential bug and get the path or condition that leads to it.

1. **Examine the Static Analysis Report:** The Clang Static Analyzer report (an HTML file) will show the code path leading to the bug. For our example, it might be trivial (it's just the call in `main`), but in complex programs, this trace can involve multiple function calls and conditions. We now have a clue *where* and *how* a bug might occur (e.g., "if input length > 9, buffer overflow occurs at this line").

2. **Use Static Analysis Information to Guide KLEE:** We have a few ways to do this:

3. *Targeted Execution:* We know the vulnerable location (e.g., the line of `strcpy` in `sample.c`). We can tell KLEE to focus on reaching this location. For instance, KLEE has an option `--exit-on-error-location=file.c:line` which will stop when an error at that location is hit ²³. We could run:

```
klee --posix-runtime --libc=uclibc \  
--exit-on-error-location=sample.c:8 \  
sample.bc --sym-arg 20
```

(Replace `sample.c:8` with the actual line number of `strcpy` in your code.) This doesn't exactly guide KLEE's search, but it ensures it halts as soon as that specific overflow is encountered (which might slightly reduce exploration of other paths).

4. *Selective Symbolic Execution:* If static analysis identified conditions for the bug (e.g., "input length >= 10 triggers overflow"), we can configure KLEE accordingly. In this simple case, we already gave `--sym-arg 20` to allow length up to 20. In more complex scenarios, you could write a harness or use `klee_assume` (if coding with KLEE API) to force certain conditions that static analysis highlighted.
5. *Instrumentation and Custom Search:* A more advanced approach from research is to **instrument the program** with the static analysis trace. For example, a tool can insert special markers or branch priorities in the code corresponding to the static analyzer's path (e.g., insert checks that follow the decisions in the static trace). Then, a custom KLEE search heuristic can prioritize those marked branches. This was demonstrated by researchers who combined Clang Static Analyzer with KLEE: they parse the CSA report, inject instrumentation calls at each step of the trace, and then run KLEE with a custom searcher that prefers following those trace steps ²⁴ ²⁵. The result is that KLEE is *guided* along the static analyzer's predicted path. This significantly prunes the search space: in their case studies, guided KLEE explored far fewer instructions and paths to reach the same bug, compared to unguided KLEE ²⁶. In fact, their guided symbolic execution confirmed bugs *more efficiently* – about **4× faster** on average in an experiment with multiple injected bugs ²⁷.

For our small example, such heavy machinery isn't needed, but in large codebases, this approach helps KLEE focus on likely bugs rather than exploring countless irrelevant paths. If static analysis says "this sequence leads to a use-after-free at line X", you can essentially tell KLEE: *try to follow this sequence*. If the bug is real, KLEE will hit it quickly and produce a test case ²². If KLEE cannot hit it, that suggests the trace

might have had assumptions that aren't feasible (potentially a false positive report), or KLEE might need more time.

1. **Run KLEE with Guidance:** Using any of the above methods, run KLEE again on the program. In our buffer overflow case, KLEE already found the bug easily. But imagine a more complex scenario where static analysis found a bug deep in a program (say, a particular input causes a null pointer dereference in a multi-step process). You could direct KLEE to that code path. For instance, you might use a combination of `--search=...` strategies or add an explicit check:
2. If using an instrumentation approach, you might run KLEE with a custom search plugin (as in the research artifact, they introduced a `step-over-minnextdistpath` searcher that uses the instrumented trace ²⁸).
3. Alternatively, use simpler means: e.g., if static analysis pinpointed a condition `X > 1000` for an overflow, you could modify the code (temporarily) to add an `if (X > 1000) klee_warning("large value");` to see if KLEE can drive X that high, etc.
4. **Interpret Results:** If the bug is real and KLEE is properly guided, you should get a test case. If KLEE does *not* hit the bug, it might mean the static analysis gave a false positive. In that case, the combination has paid off by saving you time: you know the static report might be a false alarm if even exhaustive symbolic execution (guided by the same trace) finds no feasible path. On the other hand, if KLEE finds an input, then the static warning is confirmed as a true positive with a concrete example.

Summary: Static analysis and symbolic execution are complementary: - *Static analysis* is fast and can cover the code broadly, pointing out suspicious areas (e.g., "here is a possible overflow if conditions A, B, C happen"). But it may over-approximate, so it might flag scenarios that never occur at runtime ²¹. - *Symbolic execution (KLEE)* is precise and can explore actual feasible execution paths, but it's computationally expensive and can get bogged down by path explosion in large programs. - **Combined approach:** Use static analysis to narrow down the search (focus on potential bug traces), then use KLEE to verify those traces and generate real inputs ²². The static tool provides a map, and KLEE is the vehicle to navigate that map and reach the destination (the bug) if it's reachable. Studies have shown this can greatly speed up bug-finding - KLEE confirmed static analyzer reports ~4x faster with guidance, compared to unguided exploration ²⁷. Moreover, KLEE can filter out false positives by attempting to follow the trace; if it can't, that suggests the bug might not be actually reachable, prompting further manual investigation ²².

In practice, integrating the two might involve writing scripts or using existing frameworks (like the one by Imperial College's Software Reliability Group that combines Clang Static Analyzer traces with KLEE) to automate the process ²⁴. But even manually, a workflow could be: - Run static analysis, get a list of warnings. - For each interesting warning, configure KLEE (via input constraints or targeted search) to try to hit it. - Use the concrete output from KLEE to confirm and understand the bug.

This dual approach leverages the **speed of static analysis** and the **accuracy of symbolic execution**, resulting in a more effective security analysis of programs.

6. Example Datasets of Vulnerable C/C++ Programs in LLVM-IR Format

To experiment further or evaluate the combined analysis system, you can use existing datasets of C/C++ programs with known vulnerabilities. One widely-used resource is the **Juliet Test Suite** from NIST's Software Assurance Reference Dataset (SARD):

- **Juliet Test Suite for C/C++ (Version 1.3):** Juliet is a large collection of small C/C++ programs (over 64k test cases) designed to include a wide range of common weaknesses (CWEs) ²⁹. These are organized by CWE identifier, and for each vulnerability pattern there are typically two variants: a *bad* version that contains the flaw, and a *good* version where the flaw is corrected ³⁰. Juliet 1.3 covers **118 different CWE categories** (buffer overflows, integer overflows, use-after-free, etc.) ³⁰. The suite is open-source (public domain) and is often used to evaluate static and dynamic analysis tools. You can download it from NIST SARD (it's a 146 MB package of source code) or via mirrors (there's a GitHub repository and even a HuggingFace dataset for Juliet). To use with KLEE, you would compile the Juliet test cases to LLVM bitcode. For example, each Juliet test case is a self-contained program; you can compile them with clang (some may require minor modifications to run in Linux environment or adjustments to ignore certain aspects like file I/O depending on KLEE's support). Researchers have created harnesses to run Juliet tests on dynamic tools ³¹, and you could do something similar with KLEE (perhaps using `test_juliet.py` from the Juliet dynamic project to automate feeding each test to KLEE).

- **Other Vulnerability Datasets:**

- *Synthetic benchmarks:* Besides Juliet, there are smaller collections like the RIPE benchmark (focused on buffer overflow exploitation techniques) or the old SAMATE reference small programs. These can be compiled to IR and used similarly.
- *Real-world vulnerable code:* You might also take known vulnerable open-source programs (for example, old versions of libraries with vulnerabilities, or the Toybox projects from CVE documentation) and compile them to bitcode. Keep in mind KLEE works best on smaller programs or specific functions; large programs (like an entire browser) are not feasible for full symbolic execution. But you can isolate modules or functions. For instance, the researchers in the KLEE case study took **GNU Coreutils** programs and injected bugs into them to test their approach ³² (Coreutils programs can be compiled to bitcode and are a common target for KLEE evaluations).
- *Security competitions and benchmarks:* There are archives from competitions like CGC (Cyber Grand Challenge) which have synthetic vulnerable programs, and datasets like "Metasploit HackBench" or others, but these may need adaptation for source-level analysis.

For starting out, Juliet is highly recommended due to its breadth and the fact that it has ground truth (you know which ones are supposed to have bugs). You can try running static analyzers on Juliet and then see if KLEE can find the same issues. Since Juliet programs are small, they're quite suitable for KLEE. For example, Juliet has many buffer overflow cases (CWE-121, CWE-122, CWE-126, etc.) – you can compile a few of those to bitcode and let KLEE search for out-of-bounds memory errors, then compare with Juliet's expected outcome.

(Note: When using large test suites like Juliet, consider writing automation scripts. You could batch convert all Juliet tests to bitcode (perhaps using a modified build system or `wllvm`), then run KLEE on each and collect results. Be mindful of setting `ulimit -n` (open file descriptors) higher if you run many KLEE instances in parallel ³³.)

7. How KLEE and Static Analysis Complement Each Other

Finally, it's important to understand the **synergy** between static analysis and symbolic execution in security program analysis:

- **Static Analysis** (e.g., Clang Static Analyzer, Infer, cppcheck): Works by examining code without running it. It's usually fast and can analyze large codebases, finding potential issues like null dereferences, buffer overflows, etc. Static analysis explores many paths *in a simplified way*, often using abstract interpretation or heuristics. It may report **false positives** because it errs on the side of caution – for example, “there *could* be an overflow if this value is larger than X,” even if that can't happen in practice ²¹. The advantage is it casts a wide net and finds *locations* of interest very quickly.
- **Symbolic Execution (KLEE)**: It actually “runs” the program with symbolic inputs, exploring path by path and solving constraints to find feasible inputs. It's very precise – if KLEE says “buffer overflow can happen,” it will provide the input that causes it. So it has **no false positives** (every bug reported is a real bug with a test case). However, it can have **false negatives** in the sense that it might time out or fail to explore some path due to path explosion or complex constraints. Symbolic execution is computationally expensive and doesn't scale easily to huge programs, but it excels at systematically exploring smaller pieces of code and providing concrete evidence for bugs.
- **Combined Use**: Using them together brings out the best of both:
 - Static analysis can **prioritize** and **guide** the search for bugs. Instead of blindly exploring, we give KLEE hints: “this function and path look suspicious.” This reduces KLEE's workload dramatically, as shown in research where guiding KLEE with static traces cut down execution paths from thousands to just dozens ²⁶.
 - Symbolic execution can **validate** static analysis results. If a static analyzer flags 100 possible issues, KLEE can attempt to find inputs for each. The ones KLEE confirms are true bugs (with actual inputs). The ones it cannot find inputs for might be false positives or require more complex reasoning – either way, developers can focus on the confirmed bugs first. As one study put it, the goal is to “convert a potential bug reported by a static analyzer into a concrete test input that triggers the bug” ³⁴. This makes triaging static analysis warnings much easier.
 - Together, they can achieve higher coverage. Static tools might miss some bugs that have very specific conditions, whereas KLEE can find those by brute-force exploration. Conversely, KLEE might miss some higher-level issues (like resource leaks or very deep paths) that static analysis could catch. Combining results gives a more comprehensive security assessment.

In our example: The static analyzer quickly pointed out the `strcpy` issue (perhaps even without needing to run KLEE first). KLEE then took that and produced an actual string that crashes the program (e.g., a 15-character string). If this were a real development scenario, the static analyzer's warning is now backed by a concrete test case from KLEE, which is incredibly useful for a developer to debug and fix the issue. On the

other hand, if static analysis had flagged something that KLEE cannot reproduce, that might save the developer from chasing a non-issue (or it might indicate the need for a more subtle input that KLEE couldn't easily find, which could lead to enhancing the analysis).

In summary, **KLEE + Static Analysis = Fast detection + Concrete verification**. Static analysis gives you the *where* and *what might happen*, and symbolic execution gives you the *how to make it happen*. This combination is powerful for security research and program analysis, allowing you to build a robust analysis pipeline on WSL or any Linux platform. By following the steps in this guide – installing the tools, preparing code, generating IR, running KLEE, and leveraging static analysis – you can create a workflow to catch security bugs and generate proof-of-concept inputs systematically, improving both the speed and accuracy of vulnerability detection ²² ²⁷ .

Sources:

1. KLEE Documentation – *Building KLEE with LLVM 13* ¹ ² ³⁵ ¹² (installation steps and dependencies)
2. KLEE Documentation – *Configuring Solvers and Runtime* ³⁶ ³⁷ (supported solvers, POSIX runtime, uClibc)
3. KLEE Tutorial – *Using the POSIX Runtime* ¹⁸ ¹⁷ (example of symbolic arguments and running KLEE)
4. ADA Logics Blog – *Capturing memory corruptions with KLEE* ¹⁶ (types of bugs KLEE can find)
5. Busse et al., ISSTA 2022 – *Combining Static Analysis Error Traces with Dynamic Symbolic Execution* ²² ²⁷ (benefits of guiding KLEE with static analysis, performance improvement)
6. Juliet Test Suite Documentation (NIST) ²⁹ (description of the Juliet C/C++ vulnerability test cases set with CWEs)
7. KLEE Research Artifact – *KLEE-SA (Static Analysis guidance)* ²⁴ ²⁶ (implementation details of guiding KLEE along static analyzer traces)
8. Juliet Dynamic Test Suite (GitHub) ³¹ (Juliet test cases adapted for dynamic analysis)

¹ ² ³ Building KLEE · KLEE

<https://klee-se.org/build/dependencies/>

⁴ ⁵ ⁶ ⁷ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ³³ ³⁵ ³⁶ ³⁷ LLVM 13 (recommended) · KLEE

<https://klee-se.org/build/build-llvm13/>

⁸ ⁹ Building KLEE · KLEE

<http://klee-se.org/releases/docs/v2.0/build-stp/>

¹⁶ ²⁰ Symbolic execution with KLEE: From installation and introduction to bug-finding in open source software.

<https://adalogics.com/blog/symbolic-execution-with-klee>

¹⁷ ¹⁸ Symbolic Environment · KLEE

<http://klee-se.org/tutorials/using-symbolic/>

¹⁹ ²³ ²⁴ ²⁵ ²⁶ ²⁸ Artefact for Combining Static Analysis Error Traces with Dynamic Symbolic Execution (Experience Paper) -Software Reliability Group

<https://srg.doc.ic.ac.uk/projects/klee-sa/artifact.html>

21 22 27 32 34 Combining Static Analysis Error Traces with Dynamic Symbolic Execution (Experience Paper)

<https://srg.doc.ic.ac.uk/files/papers/sa-dse-issta-22.pdf>

29 30 juliet dataset

<https://743jason.tistory.com/15>

31 GitHub - ispras/juliet-dynamic: Juliet C/C++ Dynamic Test Suite

<https://github.com/ispras/juliet-dynamic>